



Python Programming Fundamentals

Polymorphism

Programming Fundamentals Team

SETU

2026-06-12



Outline

1. What is Polymorphism?
2. Method Overriding
3. Polymorphic Behaviour
4. Duck Typing
5. Duck Typing in Practice
6. Polymorphism Summary



Outline

1. What is Polymorphism?

2. Method Overriding

3. Polymorphic Behaviour

4. Duck Typing

5. Duck Typing in Practice

6. Polymorphism Summary



1.1 Many forms, one interface

Poly = many, **morph** = form.

Polymorphism lets objects of **different types** respond to the **same method call** in their own way.

```
for user in users:
    print(user)      # calls __str__ – different output per type
```

- FreeUser.__str__ produces: "FreeUser: Alice (max 10 posts)"
- PremiumMember.__str__ produces: "PremiumMember: Bob [Gold]"

Same call (print), different behaviour — that is polymorphism.



Outline

1. What is Polymorphism?
- 2. Method Overriding**
3. Polymorphic Behaviour
4. Duck Typing
5. Duck Typing in Practice
6. Polymorphism Summary



2.1 The mechanism behind polymorphism

```
class Shape:
    def area(self) -> float:
        return 0.0
    def describe(self) -> str:
        return f"I am a shape with area {self.area():.2f}"

class Circle(Shape):
    def __init__(self, radius: float):
        self.__radius = radius
    def area(self) -> float:
        import math
        return math.pi * self.__radius ** 2

class Rectangle(Shape):
    def __init__(self, w: float, h: float):
        self.__w = w; self.__h = h
```



2.1 The mechanism behind polymorphism

```
def area(self) -> float:  
    return self.__w * self.__h
```



Outline

1. What is Polymorphism?
2. Method Overriding
- 3. Polymorphic Behaviour**
4. Duck Typing
5. Duck Typing in Practice
6. Polymorphism Summary



3. Polymorphic Behaviour

```
shapes = [  
    Circle(5),  
    Rectangle(4, 6),  
    Circle(3),  
    Rectangle(2, 8),  
]  
  
# All are Shapes – each calls its own area()  
for shape in shapes:  
    print(shape.describe())  
  
# Output:  
# I am a shape with area 78.54  
# I am a shape with area 24.00  
# I am a shape with area 28.27  
# I am a shape with area 16.00
```



3. Polymorphic Behaviour

```
total = sum(s.area() for s in shapes)
print(f"Total area: {total:.2f}")
```



Outline

1. What is Polymorphism?
2. Method Overriding
3. Polymorphic Behaviour
- 4. Duck Typing**
5. Duck Typing in Practice
6. Polymorphism Summary



4.1 Python's informal polymorphism

“If it walks like a duck and quacks like a duck, it’s a duck.”

In Python, you don’t need formal inheritance for polymorphism — you just need the right method:

```
class Dog:
    def speak(self): return "Woof!"

class Cat:
    def speak(self): return "Meow!"

class Robot:
    def speak(self): return "Beep boop."

animals = [Dog(), Cat(), Robot()] # No common parent!
```



4.1 Python's informal polymorphism

```
for a in animals:  
    print(a.speak())    # works because all have speak()
```



Outline

1. What is Polymorphism?
2. Method Overriding
3. Polymorphic Behaviour
4. Duck Typing
- 5. Duck Typing in Practice**
6. Polymorphism Summary



5. Duck Typing in Practice

```
def total_area(shapes: list) -> float:
    """Works for any object with an area() method."""
    return sum(s.area() for s in shapes)

# All of these work – no formal parent class needed
class Triangle:
    def __init__(self, base, height):
        self.__b = base; self.__h = height
    def area(self) -> float:
        return 0.5 * self.__b * self.__h

shapes = [Circle(5), Rectangle(4, 6), Triangle(3, 8)]
print(f"Total: {total_area(shapes):.2f}")
```



Outline

1. What is Polymorphism?
2. Method Overriding
3. Polymorphic Behaviour
4. Duck Typing
5. Duck Typing in Practice
- 6. Polymorphism Summary**



6. Polymorphism Summary

Concept	Description
Polymorphism	Same call, different behaviour per type
Method overriding	Child class redefines a parent method
Duck typing	Any object with the right method works
<code>isinstance()</code>	Check if an object is a specific type



Thanks for Watching - Any questions?

